# BusinessGPT: An AI-Powered Conversational Business Intelligence Platform for Indian Small and Medium Enterprises

Aditya Wagh
*Department of Computer Engineering*
*SVKM's NMIMS MPSTME Shirpur*
*Shirpur, India*
*[email address]*

***Abstract*—Small and medium enterprises (SMEs) in India generate large volumes of transactional data but rarely have the tools or expertise to turn it into business intelligence (BI). Existing BI platforms are either too complex for non-technical owners or too generic for Indian business workflows. This paper presents BusinessGPT, a conversational BI platform that lets SME owners query, monitor, and act on their business data through text, voice, and WhatsApp. BusinessGPT uses a six-layer architecture: a FastAPI service layer, a LangGraph multi-agent orchestrator, a retrieval layer combining PostgreSQL and ChromaDB, and a multi-model inference layer. Instead of sending every request to one large language model (LLM), the inference layer routes each sub-task to a cloud-hosted reasoning model on Groq, an on-device model served through Ollama for privacy-sensitive queries, or a fallback model under load, following the cost-aware routing idea studied in prior work. A proactive alert layer monitors business metrics without waiting for a user query. The paper describes the architecture, a routing algorithm, ten system capabilities, and an evaluation protocol, and compares the design with Power BI and Khatabook. The system is at the implementation stage, and a pilot evaluation is planned.**

***Index Terms*—conversational AI, business intelligence, multi-agent systems, LangGraph, retrieval-augmented generation, LLM routing, small and medium enterprises.**

## I. Introduction

Recent progress in large language models (LLMs) [1], built on the Transformer architecture [2], has greatly expanded what conversational software can do. In India, small and medium enterprises (SMEs) contribute substantially to employment and gross domestic product [3], yet many still rely on paper ledgers or disconnected digital tools to monitor performance. Commercial business intelligence (BI) platforms such as Power BI and Tableau provide analytics, but they demand technical skill and time to build dashboards, which most SME owners lack and cannot afford to acquire while running daily operations. LLMs combined with retrieval-augmented generation (RAG) [4] can close this gap by letting owners ask questions about their operations in natural language, through channels they already use.

Sending every request through a single LLM, from a basic ledger check to a complex financial forecast to a private question about unpaid vendor bills, forces a compromise among answer quality, speed, cost, and data security. A large model is costly for a simple lookup, while a small local model may lack the ability to perform detailed financial analysis. This paper introduces BusinessGPT, a BI system for SMEs that handles this trade-off with a multi-model approach: each sub-task is assigned to a suitable model according to its complexity, latency requirement, and data sensitivity. The main contributions are:

(i) A six-layer architecture that integrates orchestration, retrieval, and multi-model LLM inference in one system.
(ii) A multi-agent design on LangGraph that splits business questions into specialized sub-tasks.
(iii) A routing policy that selects the LLM for each sub-task instead of using one fixed model.
(iv) A proactive alerting function that raises flags before the user asks.
(v) A delivery system over WhatsApp and voice, channels that many small businesses already use daily.

The rest of the paper is organized as follows. Section II reviews related work and the research gap. Section III states the requirements. Sections IV–VI present the architecture, agent orchestration, and model strategy. Section VII covers implementation, Sections VIII and IX give use cases and a comparison with existing tools, Section X describes the evaluation plan, Section XI discusses limitations and ethics, and Section XII concludes.

## II. Related Work and Research Gap

### A. *Dashboard-Based BI Tools*

Platforms such as Power BI and Tableau offer visualization and drag-and-drop dashboard building. Users must still define their own metrics, build reports, and interpret the charts. Such tools are aimed at analysts who already know which questions to ask. Natural-language features have been added to some products [5], but they remain built on the dashboard paradigm rather than on messaging-based, SME-specific workflows.

### B. *Conversational BI and Bookkeeping Tools*

General-purpose conversational data-analysis tools such as Julius AI let users ask questions about their data in natural

language, but they are not built around routine SME tasks such as stock tracking, GST invoices, UPI payment checks, or supplier payments. Lightweight bookkeeping apps such as Khatabook are popular for recording transactions and producing basic ledger summaries, but they offer little proactive insight or advice.

### C. *Multi-Agent LLM Orchestration*

Graph-based agent coordination [6] and conversational multi-agent frameworks [7] split complex tasks among specialized agents. Combining reasoning with tool use, as in ReAct [8], is reported to help on tasks that need structured data retrieval and multi-step reasoning. LangGraph [9] expresses such workflows as stateful graphs, which suits the branching query flows in BusinessGPT.

### D. *Model Routing and Retrieval*

No single LLM is best on cost, speed, and accuracy at once. FrugalGPT [10] cascades queries across models of different cost, and RouteLLM [11] learns routing decisions from preference data; both report large cost reductions at similar quality. RAG [4] grounds answers in retrieved records rather than the model's internal knowledge, which reduces hallucination [12] and matters when a question needs exact figures.

### E. *Research Gap*

The reviewed tools each cover part of the SME need, but we are not aware of one that combines all four of these properties: (i) conversational access through natural language, voice, and WhatsApp; (ii) proactive alerting that does not wait for a query; (iii) multi-agent decomposition of complex questions; and (iv) multi-model routing on cost, latency, and privacy. BusinessGPT is designed to close this gap, and each property maps to an architectural component described in Sections IV–VI.

## III. Problem Statement and Requirements

The problem addressed is: how can an Indian SME owner with no technical training receive accurate, timely, and privacy-conscious answers to business questions through a platform already used every day?

### A. *Functional Requirements*

- Accept natural-language questions through text, voice, or WhatsApp.
- Answer from both structured transactional data and unstructured business information.
- Handle multi-step queries and calculations across different records.
- Notify the owner when a tracked metric crosses a set limit.
- Assign each sub-task to a model suited to its size and sensitivity.

### B. *Non-Functional Requirements*

- *Latency:* routine WhatsApp queries should be answered within a few seconds.
- *Privacy:* privacy-sensitive queries should be processed inside the local deployment whenever possible.
- *Cost efficiency:* cloud-hosted model calls should be reserved for tasks that need them.
- *Availability:* under load, the system should switch to a fallback model instead of failing.
- *Extensibility:* new agents or LLMs should be addable without redesigning the orchestration graph.

## IV. Proposed System Architecture

BusinessGPT has six layers, shown in Fig. 1: interface, API and service, orchestration, data and retrieval, multi-model inference, and proactive alerting.

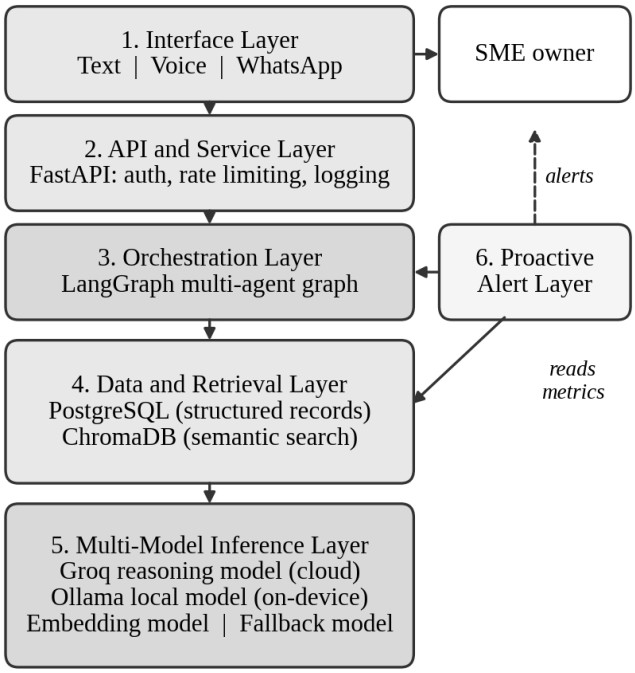



Fig. 1. Six-layer system architecture of BusinessGPT.

### A. *Interface Layer*

Users interact through text, voice commands processed by speech recognition, or WhatsApp messages sent via the WhatsApp Business Platform [13]. Each channel adapter converts its input into a common internal message format and passes it to the service layer, so owners need not learn a dashboard tool.

### B. *API and Service Layer*

A FastAPI-based service [14] exposes endpoints that receive queries from every channel. It converts each query into a standard request, attaches session and business-account information, and forwards it to the orchestration layer. It also enforces rate limiting, request logging, and authentication.

### C. *Orchestration Layer*

A LangGraph orchestrator breaks each query into intent classification, data retrieval, computation, and response formatting. Because the graph supports branching, an anomaly-

detection query follows a different path from a routine ledger lookup (Section V).

D. *Data and Retrieval Layer*

Structured business records are stored in PostgreSQL [15]. ChromaDB [16] serves as the vector store for loosely structured context such as notes, invoices, and correspondence, using embeddings from a Sentence-BERT-style encoder [17]. This lets the system combine exact lookups with semantic search.

E. *Multi-Model Inference Layer*

Instead of using one LLM, BusinessGPT keeps a pool of models and selects one per sub-task (Section VI). A single-model setup cannot satisfy the cost, latency, and privacy requirements of Section III-B at the same time, which is why this layer is central to the design.

F. *Proactive Alert Layer*

A background scheduler periodically checks business metrics against set or learned limits. When a metric leaves its normal range, the Alert Dispatch Agent sends a WhatsApp notice or places a voice call without any user query.

## V. Multi-Agent Orchestration Design

The orchestration layer is a directed graph in which each node is an agent handling one step of a query, and edges define the flow according to the state produced by each node.

A. *Agent Roles*

- *Intent Classification Agent:* decides whether a question needs a data lookup, a calculation, anomaly checking, or alert setup.
- *Retrieval Agent:* queries PostgreSQL and ChromaDB for the records and semantic context needed.
- *Computation Agent:* performs aggregation and trend forecasting using methods such as XGBoost [18] and Prophet-style time-series decomposition [19].
- *Anomaly Detection Agent:* checks whether retrieved metrics lie outside acceptable ranges using established anomaly-detection techniques [20].
- *Response Formatting Agent:* turns results into a reply suited to the user's channel (text, voice, or WhatsApp).
- *Alert Dispatch Agent:* runs independently of user queries and sends alerts when thresholds are breached.

B. *Graph Execution Flow*

A request first reaches the Intent Classification Agent, which routes lookups to the Retrieval Agent, multi-step reasoning to the Computation Agent, and anomaly-related requests to the Anomaly Detection Agent (Fig. 2). All paths converge at the Response Formatting Agent, which returns the answer to the originating channel. The flow follows the ReAct reasoning-acting pattern [8], so quick lookups finish fast while multi-step questions get the extra reasoning they need.

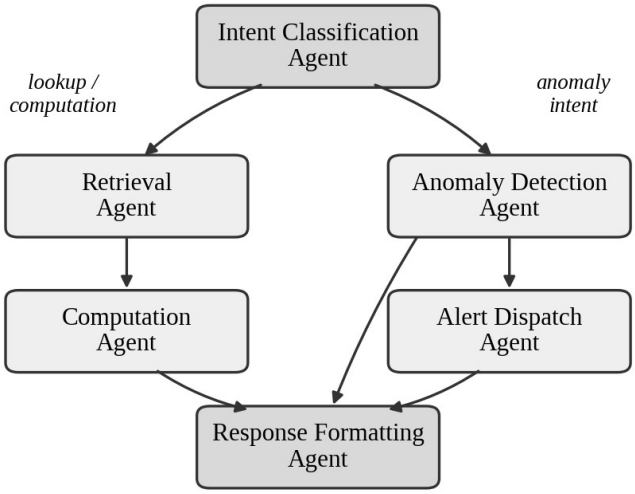



Fig. 2. Multi-agent orchestration graph implemented in LangGraph.

C. *State Management*

All agents share one graph state containing the question, the identified intent, retrieved records, intermediate calculations, and the model chosen for the current sub-task. Later agents reuse earlier results instead of repeating searches or calculations.

## VI. Multi-LLM Model Strategy

Small sub-tasks such as intent classification can be handled by a small, fast model, while difficult tasks such as detailed financial reasoning need a larger one. Queries involving sensitive financial data go to a model hosted on the user's own server so the data stays on the premises; non-sensitive quick queries go to a fast cloud endpoint.

A. *Model Pool*

Table I lists the model pool. Cloud inference runs on GroqCloud [21]. Local models run through Ollama [22] and come from open-weight families such as Llama [23].

TABLE I
MULTI-LLM MODEL POOL

| Model | Host | Primary role |
|---|---|---|
| Large reasoning model (70B class) | Groq (cloud) | Multi-step planning, financial reasoning |
| Small local model (3B–8B class) | Ollama (on-device) | Privacy-sensitive lookups, intent classification |
| Embedding model | Local or cloud | Vector encoding for ChromaDB retrieval |
| Fallback model | Groq (cloud) | Serves requests when the primary model is rate-limited |

B. *Routing Policy*

Before a sub-task is sent to an LLM, the Intent Classification Agent and the Computation Agent consult a lightweight routing policy (Algorithm 1). Sensitive sub-tasks always go to the local Ollama model, the reasoning model runs on Groq, and the fast model is a small local or cloud model; other sub-tasks go to the reasoning model or the fast model

depending on their depth. The fallback model is used only for non-sensitive sub-tasks, so that sensitive data never leaves the local deployment, even under load.

*Algorithm 1: Multi-LLM routing policy*
*Input:* sub-task t with fields t.sensitive (Boolean), t.depth (integer)
*Output:* selected model m
1: if t.sensitive then m ← LOCAL_MODEL
2: else if t.depth > DEPTH_THRESHOLD then
3: m ← REASONING_MODEL
4: else m ← FAST_MODEL
5: if not t.sensitive and m.is_rate_limited() then
6: m ← FALLBACK_MODEL
7: return m

DEPTH_THRESHOLD will be tuned on pilot query logs (Section X). If a downstream agent judges a fast-model answer inadequate, the sub-task is re-run on a larger model, so escalation can happen during a query and not only at the start.

### C. *Cost, Latency, and Privacy Trade-off*

Simple questions get low-cost answers from a small model, complex questions use the larger model's reasoning only when needed, and sensitive data stays inside the local deployment. Together these choices target the latency, cost, and privacy requirements of Section III-B.

## VII. Implementation and Deployment

### A. *Technology Stack and Capabilities*

The service layer uses FastAPI, orchestration uses LangGraph, PostgreSQL holds transactional data, and ChromaDB stores vector embeddings. Inference is split between Groq-hosted cloud endpoints and Ollama-hosted local models. Voice input relies on a weakly supervised speech recognition model [24]. BusinessGPT provides ten capabilities: natural-language query understanding, structured data ingestion, semantic document retrieval, multi-agent query decomposition, aggregation, anomaly detection, proactive alert generation, WhatsApp delivery, voice interface management, and multi-LLM routing.

### B. *Deployment and Security*

The local model runs on-site or on a low-cost edge server at the business whenever feasible, so privacy-sensitive tasks never cross the internet. Cloud-hosted components sit behind the FastAPI layer, which handles authentication and keeps the WhatsApp and voice adapters separate from the model infrastructure. Each tenant's data is isolated in the database, every endpoint requires an authenticated session token, and owners can delete their alert thresholds and business data at any time.

### C. *Testing Strategy*

Three test stages precede the pilot. Unit tests check each agent in isolation, for example that intent labels are assigned correctly and that totals are computed correctly on synthetic transactions. Integration tests run the LangGraph chain from channel input to final reply on both the lookup and anomaly paths. Regression tests re-run whenever the routing policy of Algorithm 1 changes, confirming that sensitive sub-tasks still go to the local model and that fallback works when the primary cloud endpoint is simulated as unavailable.

## VIII. Use-Case Scenarios

*Scenario 1: routine ledger lookup.* An owner asks on WhatsApp, "What were my total sales last week?" The Intent Classification Agent labels it a lookup and routes it to the local model. The Retrieval Agent pulls the transactions from PostgreSQL, the Computation Agent totals them, and the Response Formatting Agent replies on WhatsApp. No cloud model is called, so the reply is fast and cheap.

*Scenario 2: multi-step financial reasoning.* An owner asks, "Compare this month's profit margin with the previous three months and tell me if any vendor's costs are rising faster than my revenue." The request is routed to the Groq-hosted reasoning model, which breaks it into steps. The Retrieval Agent gathers several months of transaction and vendor-cost data, the Computation Agent analyzes margin trends per vendor, and the Response Formatting Agent presents the result in plain language.

*Scenario 3: proactive anomaly alert.* Without any owner query, the Alert Dispatch Agent notices that an expense category has exceeded its average by a set threshold. The Anomaly Detection Agent verifies the deviation using the local model, so detailed transaction data never leaves the deployment, and a WhatsApp alert summarizing the anomaly is sent to the owner.

## IX. Comparative Analysis

Table II compares BusinessGPT with Power BI and Khatabook on the features most relevant to Indian SMEs.

TABLE II
FEATURE COMPARISON

| Feature | BusinessGPT | Power BI | Khatabook |
|---|---|---|---|
| Conversational interface | Yes (text, voice) | Limited | No |
| WhatsApp-native access | Yes | No | No |
| Proactive alerting | Yes | Limited | Limited |
| Designed for Indian SMEs | Yes | No | Yes |
| Multi-agent orchestration | Yes | No | No |
| Multi-model LLM routing | Yes | No | No |

In principle, each row could be added to an existing tool separately: a dashboard tool could gain a chat interface, or a bookkeeping app could gain alerts. BusinessGPT differs because these properties follow from one architectural choice, namely decomposing every query into agent-level sub-tasks and routing each to the best model. Proactive alerting, for example, is not a bolt-on: the Alert Dispatch Agent runs inside the same orchestration graph (Fig. 2). The design is framed around India because WhatsApp is a primary business channel

there, but the same framework could serve SMEs elsewhere by swapping the interface adapter for the locally dominant messaging platform.

## X. Evaluation Plan

BusinessGPT is at the design and implementation stage, so the planned evaluation covers four areas: query-response accuracy against ground-truth values on curated SME transaction datasets; end-to-end latency, measured separately for locally routed and cloud-routed requests; cost per query relative to a single-model baseline; and usability, gathered from pilot SME users on WhatsApp, including their trust in the proactive alerts.

### A. *Metric Definitions*

Query-response accuracy is the fraction of test queries whose numeric or categorical answer matches the ground truth within a defined margin:

$$\text{Accuracy} = N_{\text{correct}} / N_{\text{total}} \quad (1)$$

where $N_{\text{correct}}$ is the number of correct answers and $N_{\text{total}}$ the number of test queries. For latency, both the median and the 95th-percentile response time are reported for each routing path, because the median can hide slow responses that hurt WhatsApp usability. Cost per query is the mean, over a mixed query set, of the cloud token cost plus the local compute cost:

$$C = (1/N) \Sigma (c_i + l_i) \quad (2)$$

with $c_i$ the cloud-inference token cost and $l_i$ the local-inference compute cost of query i, summed over all N queries.

### B. *Baseline for Comparison*

The routing strategy will be compared against a single-model baseline in which every sub-task, however sensitive or complex, is sent to the Groq-hosted reasoning model. This isolates the effect of the routing policy in Algorithm 1, so any difference in latency or cost can be attributed to routing rather than to the choice of model. The study is designed to test whether combining models reduces cost and delay compared with sending every request to one large model, at comparable accuracy.

## XI. Limitations and Ethical Considerations

The system has not yet been tested in a pilot, so this paper reports no measured latency, cost, or accuracy figures; these will be obtained by executing the plan in Section X. The routing policy currently decides query sensitivity with fixed rules, which may misclassify queries as real usage patterns emerge; a misclassified sensitive query could reach a cloud model, so this rule set needs particular scrutiny. LLM answers on financial data can also be wrong [12], which RAG reduces but does not remove.

The system handles data from small business owners who often have little recourse if it is mishandled. The design therefore routes financial data to local processing by default and gives owners control over what is stored and for how long. This is consistent with the data-minimization principles of the GDPR [25] and the Digital Personal Data Protection Act of India [26], although formal compliance has not been audited.

## XII. Conclusion and Future Work

BusinessGPT is a BI platform designed for Indian SME owners and the way they communicate. It combines a LangGraph multi-agent orchestrator, retrieval over structured and semantic data, multi-model LLM routing, and proactive alerts delivered on WhatsApp and voice. Future work will complete pilot deployments with SME users and measure accuracy, latency, and cost as planned in Section X, then tune the routing policy on the observed query patterns. Further plans include inventory reordering logic and privacy-preserving fine-tuning of the local model, using parameter-efficient methods such as LoRA [27] and federated learning [28] across SME deployments.

## References


[1] T. Wu et al., "A brief overview of ChatGPT: The history, status quo and potential future development," *IEEE/CAA J. Autom. Sinica*, vol. 10, no. 5, pp. 1122–1136, May 2023.

[2] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.

[3] Ministry of Micro, Small and Medium Enterprises, Government of India, "Annual report," 2023.

[4] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[5] Microsoft Corporation, "Power BI documentation," Microsoft Learn, 2024. [Online]. Available: https://learn.microsoft.com/power-bi/

[6] Y. Shen et al., "HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023.

[7] Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023.

[8] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[9] LangChain, Inc., "LangGraph documentation," 2024. [Online]. Available: https://langchain-ai.github.io/langgraph/

[10] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," arXiv:2305.05176, 2023.

[11] I. Ong et al., "RouteLLM: Learning to route LLMs with preference data," arXiv:2406.18665, 2024.

[12] Z. Ji et al., "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.

[13] Meta Platforms, Inc., "WhatsApp Business Platform documentation," 2024. [Online]. Available: https://developers.facebook.com/docs/whatsapp

[14] FastAPI, "FastAPI documentation," 2024. [Online]. Available: https://fastapi.tiangolo.com/

[15] PostgreSQL Global Development Group, "PostgreSQL 16 documentation," 2024. [Online]. Available: https://www.postgresql.org/docs/16/

[16] Chroma, Inc., "Chroma documentation," 2024. [Online]. Available: https://docs.trychroma.com/

[17] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods Nat. Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[18] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 785–794.

[19] S. J. Taylor and B. Letham, "Forecasting at scale," *Amer. Statistician*, vol. 72, no. 1, pp. 37–45, 2018.

[20] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM Comput. Surv.*, vol. 41, no. 3, pp. 1–58, 2009.
[21] Groq, Inc., “GroqCloud API documentation,” 2024. [Online]. Available: https://console.groq.com/docs
[22] Ollama, “Ollama documentation,” 2024. [Online]. Available: https://ollama.com/
[23] H. Touvron et al., “Llama 2: Open foundation and fine-tuned chat models,” arXiv:2307.09288, 2023.
[24] A. Radford et al., “Robust speech recognition via large-scale weak supervision,” arXiv:2212.04356, 2022.
[25] European Parliament and Council, “Regulation (EU) 2016/679 (General Data Protection Regulation),” *Official J. Eur. Union*, vol. L119, 2016.
[26] Government of India, “The Digital Personal Data Protection Act, 2023,” Act No. 22 of 2023.
[27] E. J. Hu et al., “LoRA: Low-rank adaptation of large language models,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.
[28] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proc. Int. Conf. Artif. Intell. Statist. (AISTATS)*, 2017, pp. 1273–1282.